From Fault to Oracle: Breaking Google Pixel TPU Hardware Memory Isolation via Trusted Deserialization Faults

Minghao Lin
mlin7282@usc.edu
University of Southern California
Los Angeles, California, USA

Guanxing Wen
guanxing.wen@certik.com
Certified Kernel Tech LLC
New York, New York, USA

Haoxuan Xu
xuhaoxua@usc.edu
University of Southern California
Los Angeles, California, USA

Yuzhou Fang
yuzhou.fang@usc.edu
University of Southern California
Los Angeles, California, USA

Nanzi Yang
nyang@odu.edu
Old Dominion University
Norfolk, Virginia, USA

Xiaokuan Zhang
xiaokuan@gmu.edu
George Mason University
Fairfax, Virginia, USA

Yanan Guo
yanan.guo@rochester.edu
University of Rochester
Rochester, New York, USA

Zhuo Zhang
zz@cs.columbia.edu
Columbia University
New York, New York, USA

Mengyuan Li
mli49061@usc.edu
University of Southern California
Los Angeles, California, USA

A Artifact Appendix

A.1 Abstract

The artifact provides a three-stage EdgeTPU firmware-analysis workflow: acquiring memory dumps on a prepared Pixel 9a; importing the dumps into IDA, recovering candidate functions, and recomputing Tables 3–4; and scanning the reconstructed firmware for candidate fault-oracle memory-access gadgets. The package includes acquisition source and binaries, two pre-collected reference logs, a firmware image, IDAPython scripts, and a prepared IDA Pro 9.2 database. Reviewers can start from the device, the supplied logs, or the prepared database. IDA and Hex-Rays are proprietary dependencies and are not bundled.

A.2 Description & Requirements

The root `README.md` documents all commands and expected output formats. `POC/DumpAuto_stable/` contains `runner`, the `cracktpu` acquisition helper, their sources, and `compile.sh`. `Public_artifacts/` contains the firmware, the pre-collected dump logs, the importer, the function-recovery script, `table3.py`, `table4.py`, and the scanner. The prepared database is `edgetpu-rio_eval_re.fw.i64` at the package root.

A.2.1 Security, privacy, and ethical concerns. Acquisition requires root, stops the EdgeTPU logging service, and can reboot the device or interrupt SSH. Use the prepared review device or an authorized test device. After reconnecting, re-enter root and stop logging before resuming; restore logging afterward. Import and recovery modify the database, so use a working copy. Scanner reports are candidates, not automatically confirmed vulnerabilities.

A.2.2 How to access. The artifact is released at https://github.com/sept-usc/From-Fault-to-Oracle. Remote access to the prepared Pixel 9a was offered only during artifact evaluation and is not part of the release. Offline stages need no device connection.

A.2.3 Hardware dependencies. Live acquisition uses the authors' prepared, rooted Pixel 9a with access to `/dev/edgetpu-soc`; this physical device is accessed remotely and shared among evaluators. Other devices or firmware builds are not assumed compatible. Offline evaluation uses a workstation capable of running IDA Pro 9.2.

A.2.4 Software dependencies. IDA Pro 9.2 with IDAPython is required; Stage 3 and the decompilation metric of Table 4 also need a licensed ARM Hex-Rays decompiler (without it, that metric is N/A). Remote access requires Tailscale and SSH. Rebuilding requires the Android NDK; deploying to a locally connected device requires ADB. The build script uses `darwin-x86_64` and `aarch64-linux-android34-clang`; adjust the NDK host path on other operating systems. SSH access does not imply ADB access.

A.2.5 Benchmarks. Inputs are `Public_artifacts/edgetpu-rio.fw`, the prepared database, and two pre-collected logs: `0x0ff00000_0x0ff0E4FF_dump.log` (14,656 words; 2,175 tagged) and `0x10000000_0x1000ea80_dump.log` (15,009 words; 2,150 tagged). Records contain an address and a 32-bit value, optionally tagged `cache_unstable` or, for words rebuilt by the neighboring-address fallback, `flag_insurance_operation_*`. These counts describe bundled inputs; unstable locations need not match byte for byte in a fresh acquisition.

A.3 Set Up

A.3.1 Installation. Extract the package and install the dependencies above. For device acquisition, the helpers are already deployed on the prepared Pixel and prebuilt copies ship in `POC/DumpAuto_stable/`, so no build is normally needed. To rebuild, set `NDK` (we used r26.1.10909125) and run `sh compile.sh` in `POC/DumpAuto_stable/`. The script runs `adb push` after each build; without ADB, which Tailscale access does not provide, both pushes fail, but the binaries are still built. Copy them with `scp -P 8022` to the SSH user's home directory; then, from the root shell, place them in `/data/local/tmp/` and make them executable. README Section 1.2 gives both deployment paths and the checksums of the bundled binaries. After logging in to the prepared Pixel via SSH,

run su to enter a root shell, then run stop edgetpu_logging; enter /data/local/tmp/, make both helpers executable, and create review-dumps/. The firmware uses 32-bit ARM in little-endian mode. For a fresh IDA import, select ARM little-endian, set the instruction set to **32-bit**, and load the firmware at **base address 0x0ffff000**. Alternatively, open a copy of the prepared .i64 directly; do not rebase or reimport it.

A.3.2 Basic test. For the shortest offline check, open the prepared database, wait for analysis, and execute Public_artifacts/faultoracle_scanner_mc.py via File > Script file. The IDA output window should show per-handler reports and end with TOTAL sites (after per-def-site dedup): .... Check for no function at or gen_microcode failed; a final total alone does not prove that every handler was analyzed.

A.4 Evaluation Workflow

A.4.1 Major claims. Section, table, and figure numbers in this appendix refer to the paper; Table 1 belongs to this appendix.

- (C1) The acquisition workflow recovers the two memory regions and supports resumption after interruption (E1). Paper: §6.2 (RQ1).
- (C2) Imported dump words support firmware reconstruction and heuristic recovery of candidate function boundaries; the supplied inputs reproduce the Table 3 counts and yield Table 4 metrics consistent with the paper (E2). Paper: §6.2, §6.4.
- (C3) The microcode scanner identifies candidate fault-oracle memory-access sites for manual inspection (E3). Paper: §6.3 (RQ2).

Table 1: Mapping between artifact experiments and the paper.

Exp.	Paper section	Results reported there
E1	§6.2 (RQ1)	Tables 2, 3, 4, 6; Appendices C, D
E2	§6.2, §6.4	Tables 3, 4; Figure 3; Appendices C, D
E3	§6.3 (RQ2)	Table 5

A.4.2 Experiments. E1 is deliberately throttled. The review device is shared among evaluators, and frequent acquisition requests, particularly under concurrent access, could occasionally destabilize it; runner therefore waits 2 s between successive reads of a word. With at least three reads per word, a complete acquisition takes over 16 h per region even without retries or reboots. Table 6 reports unthrottled costs, with one read per normal word. E2 and E3 start from the pre-collected logs and the prepared database, so reviewers need not wait for E1.

- (E1) **Memory acquisition (C1; README Stage 1).** *Paper:* §6.2; Region 1 in Table 4 and Appendix D, Region 2 in Tables 2 and 3 and Appendix C, cost in Table 6. *Preparation:* Connect, enter root, and stop logging as above. *Execution:* In /data/local/tmp/, invoke ./runner START END ./cracktpu OUTPUT twice: first with START=0x0ff00000, END=0x0ff0e500; then with START=0x10000000, END=0x1000ea84. Use the matching log names under review-dumps/. END is exclusive and steps are four bytes. A word is logged after three consecutive matching reads; after 15 attempts, runner falls back to neighboring-address reconstruction (the paper's sliding-offset fallback). After reboot, reconnect, repeat su and stop edgetpu_logging, and rerun the same command/output path. Runner resumes after the last valid record; use a new output path for an independent run, as the shared device may hold other evaluators' logs. Optionally, run ./cracktpu 0x10000000 once for a timed read. Finish with start edgetpu_logging. *Results:* Retrieve logs and check their final word addresses: 0x0ff0e4fc and 0x1000ea80, respectively. Inspect tagged records and incomplete acquisitions rather than requiring identical values at every location. The direct run's [TIME] line gives Table 6's per-read cost.
- (E2) **Import, recovery, Tables 3–4 (C2; README Stage 2).** *Paper:* §6.2 (Tables 3 and 4, Appendices C and D); §6.4 (Figure 3). *Preparation:* Load the firmware at the stated base and save a copy. *Execution:* Run import_dump_log_to_ida.py twice, selecting one supplied log each time; wait for reanalysis. Run recover_tpu_funcs.py, checking that SEG_NAME (default ROM) names the intended segment, e.g., .dump_0ff (DUMP_ROM in the prepared database); repeat for other relevant segments as needed. Then open a fresh copy of the prepared database and run table3.py and table4.py; each prints an aligned table in IDA's Output window. *Results:* Expect 14,656 and 15,009 patched words if all addresses are mapped; investigate malformed/unmapped records. Recovery output should contain DONE — added ... funcs total; zero additions may reflect prior recovery or a wrong segment. With IDA and Hex-Rays 9.2, table3.py reports the pattern counts 92, 9, 39, and 5, matching Table 3, and table4.py reports 98.96%, 99.52%, 98.66%, 98.18%, and 98.98%. Because recover_tpu_funcs.py was reimplemented for AE, these rates differ slightly from Table 4 but support the paper's conclusion that the recovered firmware is amenable to structural analysis.
- (E3) **Fault-oracle scanning (C3; README Stage 3).** *Paper:* §6.3 (Table 5); scanner design in §5. *Preparation:* Use the prepared database and ARM decompiler. *Execution:* Run the scanner as in the basic test; it analyzes configured HANDLERS at MMAT_LOCOPT maturity. *Results:* Save IDA/Hex-Rays versions, database identity, and per-handler output, including site addresses and deduplicated counts. Only Table 5's *Reported* column is automated; *Structurally valid* and *Reachable* are the manual audit of §6.3. Inspect reported instructions and pseudocode. Sites are fault-oracle candidates, not conventional ROP gadgets or proof of exploitability.

A.5 Notes on Reusability

The pre-collected logs and the prepared database allow offline evaluation without the Pixel. New firmware versions require reviewing segment names, handler addresses, and payload registers. Function recovery is heuristic; scanner failures must not be interpreted as absence of candidates.

A.6 Version

Based on the LaTeX template for artifact evaluation version 1.0.